Machine Learning on VLSI

Proyecto Final

Alumno:

Manuel Exequiel Arias

12 de mayo de 2026

Índice

1. Enunciado del proyecto **3**
1.1. Objetivo . . . 3
1.2. Dataset . . . 3
1.3. Problema a resolver . . . 3
1.4. Desarrollo del modelo . . . 3
1.5. Cuantización del modelo . . . 4
1.6. Análisis para implementación en hardware . . . 4
1.7. Resultados esperados . . . 5

2. Desarrollo **6**
2.1. Análisis y Preprocesamiento del Dataset . . . 6
2.1.1. Estructura de Datos . . . 6
2.1.2. Normalización de Amplitud . . . 6
2.2. Desarrollo y Entrenamiento del Modelo . . . 7
2.2.1. Diseño de la Arquitectura de la Red Neuronal . . . 7
2.2.2. Procedimiento de Entrenamiento . . . 8
2.2.3. Metodología de Evaluación y Métricas de Desempeño . . . 8
2.3. Cuantización del Modelo . . . 9
2.3.1. Análisis Estadístico de la Distribución de Parámetros . . . 9
2.3.2. Definición del Formato de Punto Fijo . . . 10
2.3.3. Transferencia de Pesos y Entrenamiento Consciente de Cuantización (QAT) . . . 11
2.3.4. Evaluación Comparativa de Rendimiento . . . 11
2.4. Análisis para Implementación en Hardware . . . 13
2.4.1. Estimación Energética mediante QTools . . . 13
2.4.2. Perfilado de Operaciones y Huella de Memoria . . . 13
2.4.3. Métricas Globales y Medida de Desempeño (Figure of Merit) 13
2.5. Resultados y Discusión: Impacto de la Cuantización . . . 14
2.5.1. Impacto sobre la Precisión (Accuracy vs. SNR) . . . 14
2.5.2. Impacto sobre la Estimación de Energía y Eficiencia . . . 15

1. Enunciado del proyecto

1.1. Objetivo

El objetivo del proyecto es diseñar, entrenar y analizar una red neuronal para resolver un problema de clasificación utilizando datos reales de señales de comunicaciones. Posteriormente, el modelo deberá optimizarse mediante técnicas de cuantización para evaluar su impacto en precisión, complejidad computacional y estimación de consumo energético.
El proyecto busca comprender el compromiso entre precisión del modelo y eficiencia de hardware, concepto central en el diseño de sistemas de inteligencia artificial para dispositivos de recursos limitados.

1.2. Dataset

El dataset utilizado es el RadioML 2016.10A, que contiene señales de radio simuladas representadas mediante muestras I/Q correspondientes a diferentes tipos de modulación y niveles de SNR (Signal-to-Noise Ratio).
Cada muestra contiene:

- 128 muestras complejas (I/Q)
- diferentes niveles de SNR
- distintos tipos de modulación

Cada ejemplo del dataset tiene dimensión: 2 x 128, donde la primera fila corresponde a la componente I y la segunda a la componente Q.

1.3. Problema a resolver

Se plantea un problema de clasificación supervisada.

- Entrada del modelo: Señal I/Q (2 x 128)
- Salida del modelo: Tipo de modulación de la señal.

El modelo deberá aprender a identificar automáticamente la modulación incluso en presencia de ruido.

1.4. Desarrollo del modelo

Tareas a realizar:

1. Analizar el dataset.
2. Preprocesar los datos.

3. Diseñar una red neuronal para el problema planteado y teniendo en cuenta la posterior implementación.
4. Entrenar el modelo utilizando TensorFlow / Keras.

El modelo podrá ser por ejemplo:

- Red totalmente conectada (MLP)
- CNN 1D
- CNN 2D

Se deberán evaluar métricas como:

- Accuracy
- Matriz de confusión
- Desempeño en función del SNR

1.5. Cuantización del modelo

Una vez entrenado el modelo en punto flotante, se deberá aplicar cuantización utilizando QKeras. Antes de definir el ancho de bits, se deberá realizar un análisis de la distribución de datos y pesos, con el fin de elegir de forma adecuada el rango de cuantización. El objetivo es analizar el impacto del ancho de bits en:

- precisión del modelo
- tamaño del modelo
- estimación de consumo energético.

1.6. Análisis para implementación en hardware

Utilizando herramientas de análisis de QKeras / QTools, se deberá estimar:

- número de operaciones MAC
- tamaño del modelo
- número de accesos a memoria
- estimación de consumo energético

Se deberá comparar el modelo original y el modelo cuantizado.

1.7. Resultados esperados

Se deberán comparar diferentes versiones del modelo:

- Modelo en punto flotante
- Modelo cuantizado (distintos bitwidth)

Se deberá analizar el impacto de la cuantización sobre:

- precisión
- tamaño del modelo
- estimación de energía

2. Desarrollo

2.1. Análisis y Preprocesamiento del Dataset

2.1.1. Estructura de Datos

El conjunto de datos RadioML 2016.10A se presenta originalmente estructurado mediante un diccionario, cuyas claves son tuplas que indican el tipo de modulación y la Relación Señal a Ruido (SNR) expresada en decibeles (dB). Para cada una de estas combinaciones, el dataset provee un conjunto de señales, donde cada muestra individual está representada por una matriz de dimensión 2×128. Estas dos filas corresponden a los canales In-Phase (Canal I o En Fase) y Quadrature (Canal Q o Cuadratura) de la señal de radiofrecuencia, mientras que las 128 columnas representan las muestras discretas en el dominio del tiempo.
A continuación, se grafica una señal de ejemplo donde se ven ambas componentes I y Q:

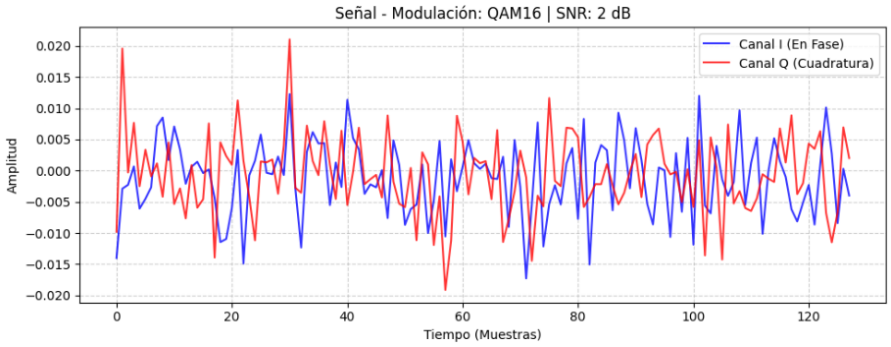
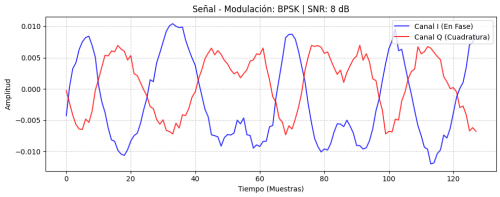


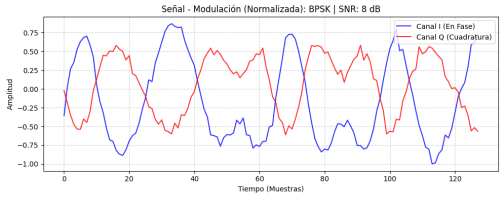
Figura 2.1: Ejemplo de señal I/Q del dataset RadioML 2016.10A (Modulación = QAM16, SNR = 2 dB).

2.1.2. Normalización de Amplitud

Para optimizar la convergencia de los algoritmos de optimización durante el entrenamiento (como Adam) y evitar inestabilidades numéricas, se aplicó un proceso de normalización independiente a cada señal. Se identificó el valor absoluto máximo presente en cada matriz de 2×128 y se dividió la totalidad de la señal por dicho escalar. Este procedimiento escala las amplitudes de los canales I y Q al intervalo $[-1, 1]$. En las siguientes figuras se muestra la señal antes y después de normalizar, respectivamente:



(a) Sin normalización



(b) Con normalización

Figura 2.2: Comparación de señales I/Q antes y después de la normalización.

2.2. Desarrollo y Entrenamiento del Modelo

2.2.1. Diseño de la Arquitectura de la Red Neuronal

Para resolver el problema de clasificación de las señales I/Q representadas en matrices de 2×128, se diseñó una red neuronal convolucional unidimensional (CNN 1D) con arquitectura residual (ResNet). La arquitectura está compuesta por aproximadamente $127,000$ parámetros entrenables y se estructura de la siguiente manera:

- **Capa Inicial:** Una capa convolucional (`Conv1D`) con 64 filtros, tamaño de kernel de 7 y *stride* de 1, seguida de normalización por lotes (`Batch Normalization`) y activación ReLU.
- **Bloque Residual 1:** Compuesto por dos capas convolucionales de 64 filtros y tamaño de kernel de 5, cada una con normalización por lotes y activación ReLU. La salida de este bloque se suma de forma directa a la entrada del mismo a través de una conexión de salto (*skip connection*), seguida de una operación de `MaxPooling1D` para reducir la dimensionalidad espacial a la mitad.
- **Bloque Residual 2:** Compuesto por dos capas convolucionales de 128 filtros y tamaño de kernel de 3. Debido al cambio en el número de canales, la conexión de salto utiliza una convolución adaptadora (*kernel* de 1×1) para igualar dimensiones antes de la suma. Seguido de un `MaxPooling1D`.
- **Capa de Salida:** Una operación de agrupamiento global promedio (`Global AveragePooling1D`) que aplana las características para conectarlas a una capa densa (`Dense`) final de 11 neuronas con activación `Softmax`, correspondiente a las 11 clases del problema.

Las arquitecturas residuales se caracterizan por la introducción de estas conexiones de salto, lo que facilita el entrenamiento de redes profundas al evitar el desvanecimiento del gradiente. En contraste con un modelo MLP tradicional, la CNN 1D captura la dependencia temporal de la señal, comparte pesos a lo largo del eje de tiempo y reduce el costo de operaciones MAC, lo que resulta más adecuado para su posterior implementación en hardware.

2.2.2. Procedimiento de Entrenamiento

El modelo fue entrenado utilizando el framework *Keras* con *TensorFlow* como *backend*. Se utilizó el optimizador **Adam** configurado con una tasa de aprendizaje inicial de 0,01 y la función de pérdida `sparse_categorical_crossentropy`. El entrenamiento se estructuró con un *batch size* de 32 muestras y se permitió un máximo de épocas suficientes para alcanzar la convergencia, la cual fue controlada mediante un algoritmo de *Early Stopping* (detención temprana). Este monitorizaba la pérdida en el conjunto de validación (`val_loss`) con una paciencia (*patience*) de 3 épocas, restaurando automáticamente los mejores pesos obtenidos para prevenir el sobreajuste (*overfitting*).

2.2.3. Metodología de Evaluación y Métricas de Desempeño

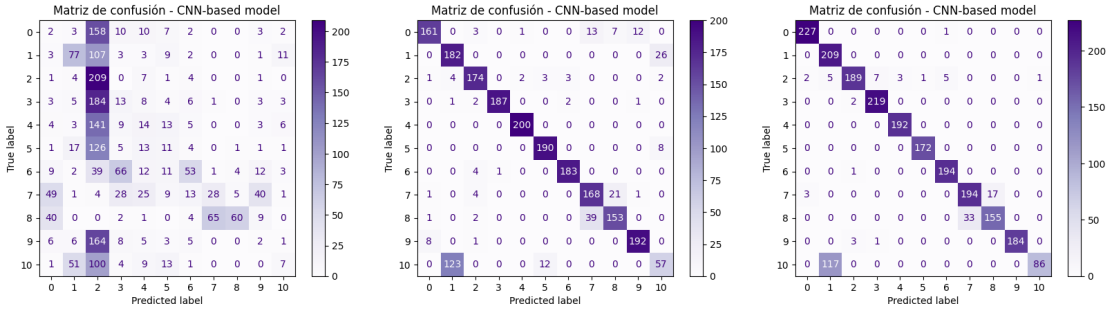
Previo al entrenamiento, el dataset completo de $220,000$ muestras fue particionado aleatoriamente empleando un 80 % para el conjunto de entrenamiento y un 20 % para el conjunto de prueba (evaluación final). De los datos de entrenamiento, un 20 % interno se separó dinámicamente como conjunto de validación durante el proceso de entrenamiento (*training phase*).

El dataset se caracteriza por estar perfectamente balanceado, conteniendo $1,000$ señales por cada combinación de Modulación/SNR. Se consideran 11 clases de modulaciones, cuya correspondencia entre la etiqueta numérica procesada por la red y el esquema de modulación físico se detalla a continuación:

- **0:** 8PSK
- **1:** AM-DSB
- **2:** AM-SSB
- **3:** BPSK
- **4:** CPFSK
- **5:** GFSK
- **6:** PAM4
- **7:** QAM16
- **8:** QAM64
- **9:** QPSK
- **10:** WBFM

Cabe destacar que durante el entrenamiento los distintos niveles de SNR fueron mezclados, obligando al modelo a extraer características intrínsecas de las señales independientemente del nivel de ruido presente. Por razones prácticas y computacionales, el análisis principal se basa en una única ejecución experimental (*single run*) del modelo convergido.

Tras el entrenamiento, el modelo fue sometido a un análisis predictivo sobre el conjunto de prueba. Para aislar el desempeño de la red de la degradación absoluta de la señal, se analizó una **Matriz de Confusión** filtrada específicamente para niveles de SNR críticos. Este análisis permitió observar las confusiones intrínsecas del modelo entre modulaciones analógicamente similares sin la contaminación del ruido blanco excesivo.



(a) SNR = -10 dB (b) SNR = 0 dB (c) SNR = +10 dB

Figura 2.3: Matriz de Confusión para diferentes niveles de SNR.

Finalmente, se trazó una **Curva de Precisión en función del SNR**. El gráfico evidencia el límite teórico del sistema: para SNRs inferiores a 0 dB, el modelo tiende a una precisión aleatoria ($\approx 9\,\%$), dado que las señales se encuentran enmascaradas por el ruido. Sin embargo, a medida que la relación señal a ruido cruza el umbral de los 0 dB a +5 dB, la precisión del modelo experimenta un crecimiento asintótico, estabilizándose en sus valores máximos de *accuracy* para los registros más limpios del dataset.

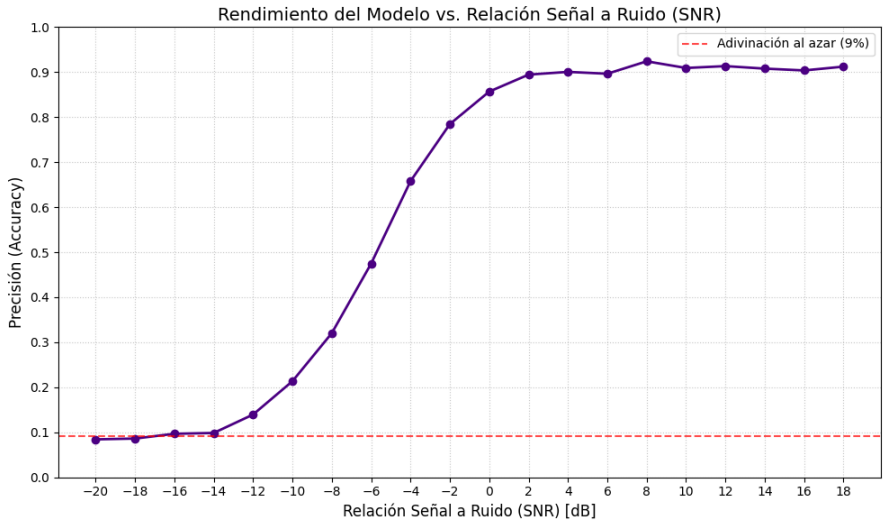


Figura 2.4: Curva de Precisión en función del SNR.

2.3. Cuantización del Modelo

2.3.1. Análisis Estadístico de la Distribución de Parámetros

El proceso de cuantización transforma la representación aritmética de los parámetros de punto flotante de 32 bits a aritmética de punto fijo, reduciendo significativamente el consumo de memoria y la complejidad lógica de los bloques multiplicadores-acumuladores (MAC). Para evitar severos errores de truncamiento (*clipping*) o

pérdida de resolución fraccionaria, se realizó un análisis estadístico previo de los pesos (*kernels*) y sesgos (*biases*) extraídos del modelo base convergido.

El análisis de los histogramas reveló comportamientos marcadamente asimétricos entre los distintos tensores. Los *kernels* de las capas convolucionales y de la capa densa exhibieron distribuciones de tipo normal con colas extendidas, alcanzando valores cercanos a ± 16. Por el contrario, los sesgos de las operaciones convolucionales mostraron una varianza mínima, concentrándose en el rango $[-1, 1]$, mientras que el sesgo de la capa de salida presentó picos que superaban ligeramente este límite unitario.

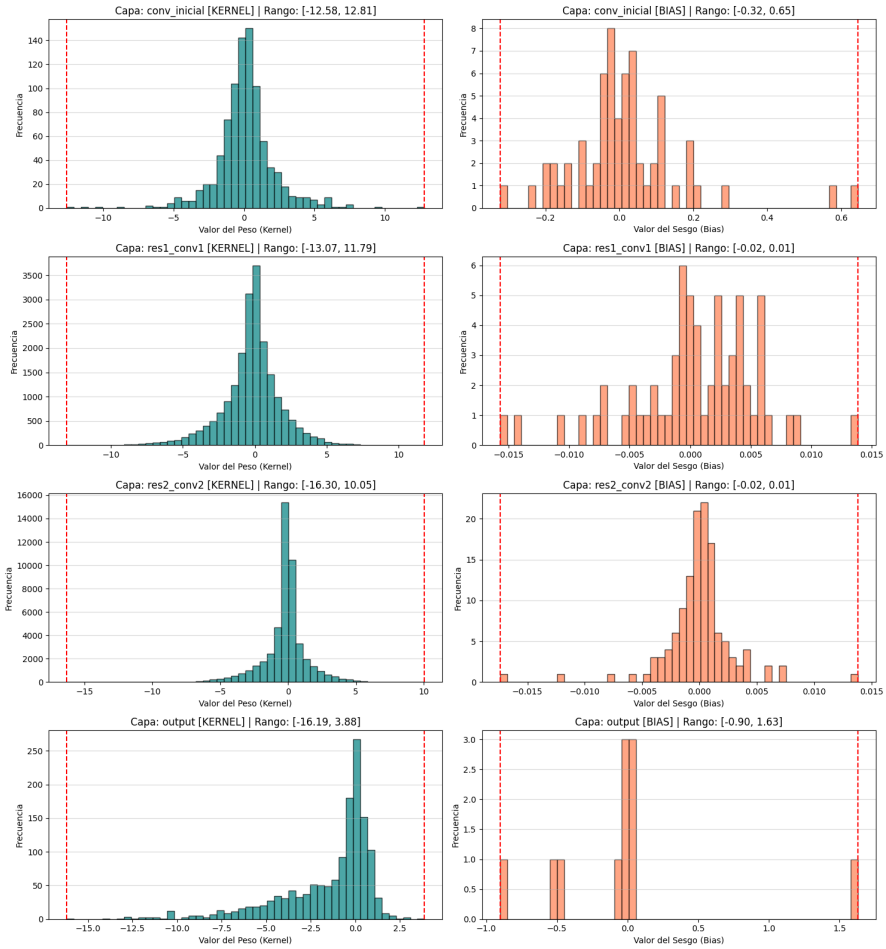


Figura 2.5: Histograma de pesos (kernels) y sesgos (biases).

Figura 2.6: Análisis estadístico de la distribución de pesos y sesgos del modelo base.

2.3.2. Definición del Formato de Punto Fijo

Con base en el análisis analítico de los rangos dinámicos, se configuraron estrategias de cuantización utilizando la herramienta QKeras. Para balancear ade-

cuadamente el error introducido y el área requerida en hardware, se definieron longitudes de palabra globales de 8 bits (y pruebas agresivas en 4 bits) para las activaciones y pesos.

Además, tal como lo exige el diseño digital estándar, la acumulación (operación MAC) asume registros de 32 bits para evitar el desbordamiento numérico (*overflow*). A continuación, en la Tabla 1, se resume la configuración de cuantización implementada para las capas convolucionales, activaciones y el formato lógico del acumulador:

Cuadro 1: Configuración de cuantización por componentes de la red (Modelo de 8 bits)

Componente / Capa	**Weight Bits (Kernels)**	**Activation Bits**	**Accumulator Bits**
Capas Convolucionales (`Conv1D`)	8 bits (4 enteros)	-	32 bits
Capas Densas (`Dense`)	8 bits (4 enteros)	-	32 bits
Activaciones internas (`ReLU`)	-	8 bits (3 enteros)	-
Sesgos (`Biases`)	8 bits (0 enteros)	-	-

2.3.3. Transferencia de Pesos y Entrenamiento Consciente de Cuantización (QAT)

Tras compilar el diccionario de cuantización, se instanció el nuevo modelo trasladando directamente los pesos previamente aprendidos en punto flotante (*Weight Transfer*). Debido a que la discretización abrupta de los valores induce un ruido de cuantización que degrada el *accuracy* inicial, el modelo fue sometido a una fase de calibración o Ajuste Fino (*Fine-Tuning*).

2.3.4. Evaluación Comparativa de Rendimiento

El análisis de validación demostró la robustez intrínseca de la arquitectura propuesta frente a la reducción de precisión. Al evaluar las matrices de confusión para escenarios específicos, como una SNR de 8 dB, se constató que la topología de falsos positivos (por ejemplo, la clásica superposición entre WBFM y AM-DSB) se mantiene estructuralmente idéntica a la del modelo original en *Float32*.

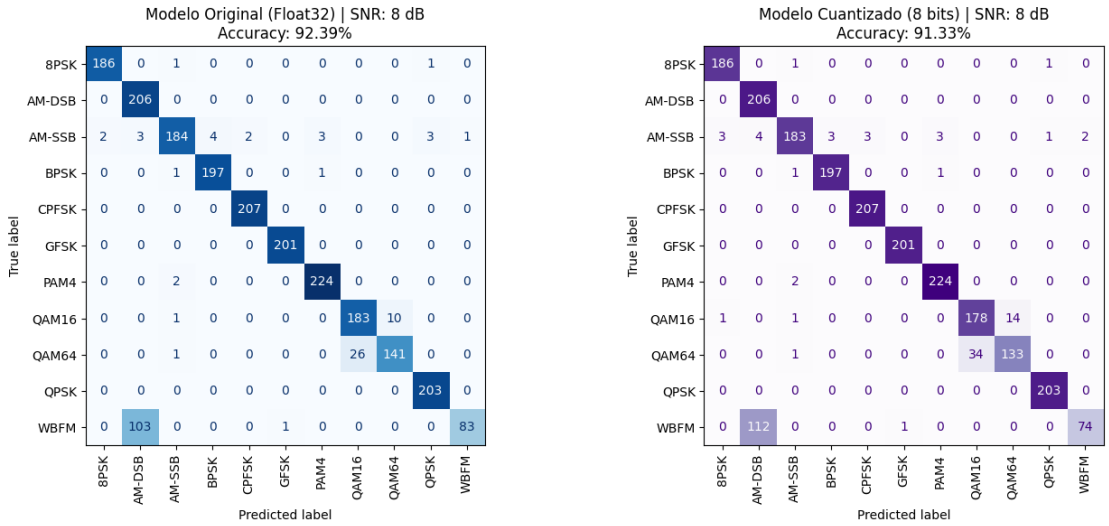


Figura 2.7: Matriz de Confusión para el modelo original y cuantizado.

Finalmente, la evaluación asintótica expresada en la curva de *Accuracy vs. SNR* verifica empíricamente el éxito del codiseño. La superposición casi perfecta entre el desempeño del modelo continuo y el modelo cuantizado evidencia que es factible reducir el costo computacional y el consumo energético sin penalizar la sensibilidad de la red neuronal.

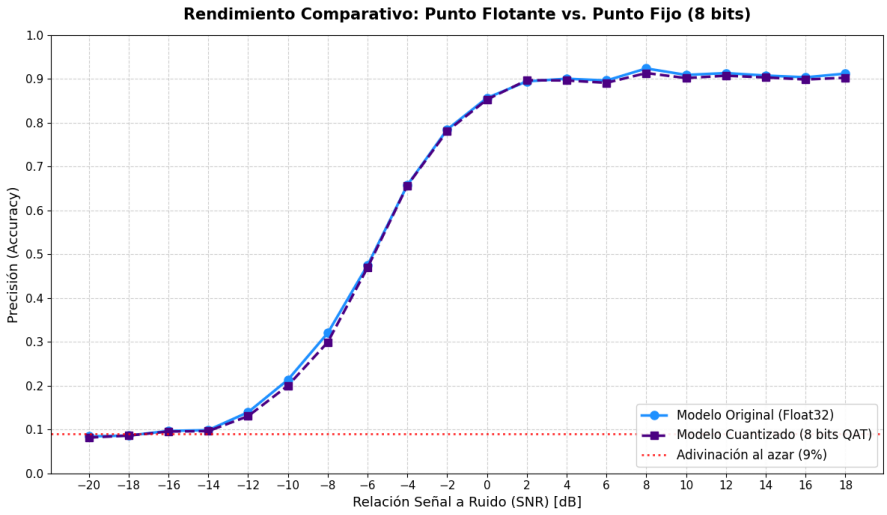


Figura 2.8: Curva de Precisión en función del SNR para el modelo original y cuantizado.

2.4. Análisis para Implementación en Hardware

2.4.1. Estimación Energética mediante QTools

Para validar la viabilidad física del modelo y cuantificar el impacto del codiseño hardware-software, se realizó una estimación del consumo energético de la fase de inferencia utilizando `QTools`. El análisis se basó en el modelo de energía del proceso de 45 nm propuesto por Horowitz (2014), el cual tabula el costo en picojulios (pJ) de las distintas operaciones aritméticas y accesos a memoria en función del ancho de bits.

Para modelar los bloques multiplicadores-acumuladores (MAC) físicos de manera realista, se estipuló el uso de acumuladores de 32 bits (`int32`). Esto resulta crítico en implementaciones VLSI, ya que la sumatoria iterativa de productos de 8 bits en registros estrechos provocaría desbordamientos aritméticos (*overflow*).

2.4.2. Perfilado de Operaciones y Huella de Memoria

La red requiere un total de $10,601,856$ operaciones MAC (multiplicaciones con signo de 8 bits, denotadas como `smult_8_8`) para procesar una ventana temporal completa de la señal I/Q.

En términos de almacenamiento, la consolidación de todos los pesos y sesgos cuantizados resulta en una huella de memoria estática de $1,006,168$ bits (aproximadamente 125,77 KB). Adicionalmente, el modelo presentó una escasez natural (*sparsity*) del 2,77 %.

2.4.3. Métricas Globales y Medida de Desempeño (Figure of Merit)

El consumo total estimado de la arquitectura ascendió a $2,999,833{,}0$ pJ (aproximadamente 3,0 μJ) por inferencia. Para contextualizar este gasto energético frente a la capacidad predictiva de la red, se definió una Medida de Desempeño o *Figure of Merit* (FoM). Esta métrica relaciona linealmente la precisión final del clasificador (*Accuracy*) con la energía invertida, expresándose en unidades de acierto por picojulio (Acc/pJ).

En la Tabla 2 se presenta un resumen de los resultados obtenidos:

Cuadro 2: Resumen de Hardware y Desempeño del Modelo ResNet-1D (8 bits)

Métrica de Hardware	**Valor Estimado**
Precisión (*Accuracy*)	0,5930
Operaciones MAC Totales	$10,601,856$
Huella de Memoria (*Weights + Biases*)	$1,006,168$ bits ($\sim$ 126 KB)
Índice de Escasez (*Sparsity*)	2,77 %
Energía Total por Inferencia	$2,999,833$,0 pJ ($\sim$ 3,0 μJ)
Medida de Desempeño (FoM)	$\mathbf{1{,}9768 \times 10^{-7}}$ Acc/pJ

2.5. Resultados y Discusión: Impacto de la Cuantización

Para consolidar el análisis del codiseño hardware-software, se llevó a cabo una evaluación comparativa sistemática entre tres iteraciones del modelo: la arquitectura original en punto flotante continuo (*Float32*), una versión cuantizada a 8 bits y una versión sometida a compresión extrema de 4 bits. Ambas versiones discretas fueron optimizadas mediante Entrenamiento Consciente de Cuantización (QAT) para garantizar una comparación justa. El impacto de la reducción del ancho de palabra (*bitwidth*) se analizó a través de tres ejes críticos: precisión algorítmica, huella de memoria y consumo energético.

2.5.1. Impacto sobre la Precisión (Accuracy vs. SNR)

La degradación del rendimiento se evaluó parametrizando la métrica de *accuracy* en función de la Relación Señal a Ruido (SNR) del canal de comunicaciones, barriendo el espectro desde -20 dB hasta $+18$ dB.

Como se observa en las curvas de rendimiento, la versión de 8 bits exhibe excelente comportamiento, superponiéndose de manera casi idéntica a la curva asintótica del modelo *Float32* (alcanzando precisiones cercanas al 80 % en alta SNR). Esto demuestra empíricamente que una resolución de 8 bits posee suficiente rango dinámico para capturar los patrones de fase y amplitud sin pérdida de información estructural.

En contraste, el modelo de 4 bits sufre una penalización evidente. A pesar de haber sido sometido a un proceso de calibración (QAT), la red satura prematuramente en la región de alto SNR. El impacto de la cuantización afecta principalmente a las modulaciones de orden superior (QAM16 y QAM64), que dependen de la precisión de los pesos para discriminar entre constelaciones densas.

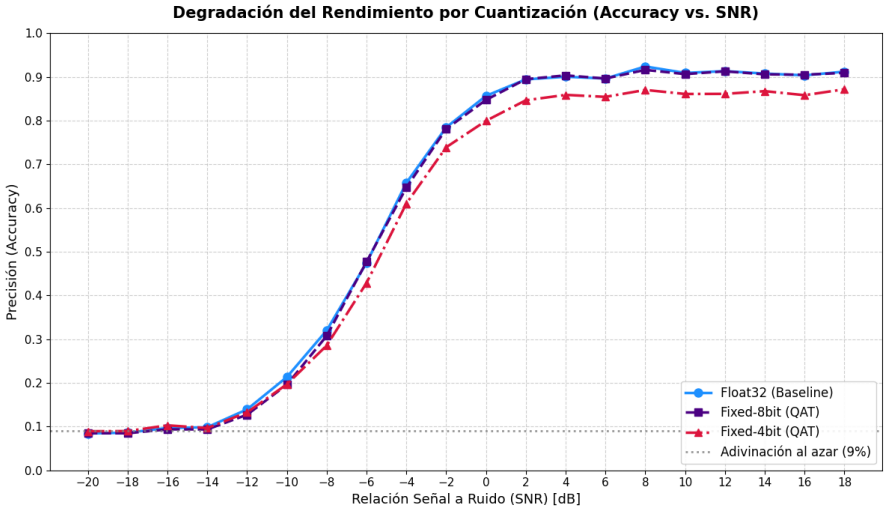


Figura 2.9: Curva de Precisión en función del SNR para el modelo original y ambas cuantizaciones (8 y 4 bits).

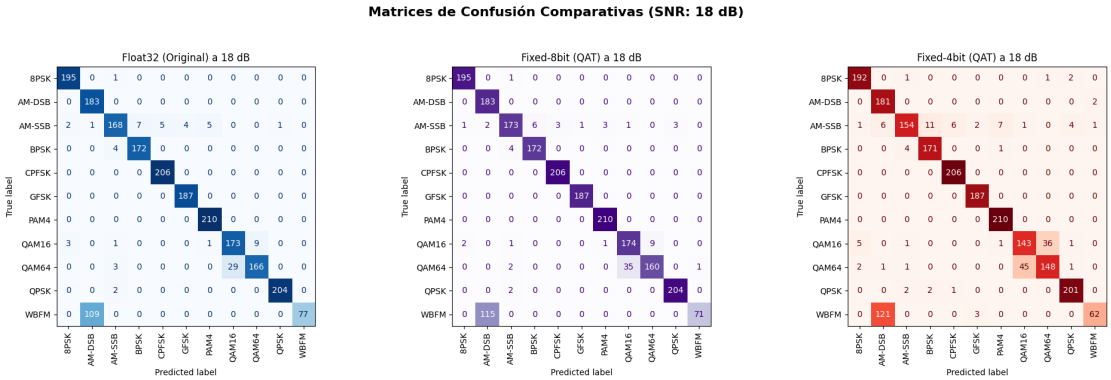


Figura 2.10: Matrices de Confusión Comparativas para el modelo original y ambas cuantizaciones (8 y 4 bits).

2.5.2. Impacto sobre la Estimación de Energía y Eficiencia

El costo energético de las operaciones de Multiplicación y Acumulación (MAC) decrece de forma cuadrática frente a la reducción del ancho de los operandos. Mientras que una inferencia en *Float32* requiere un consumo masivo debido a la complejidad de las unidades de coma flotante, la estimación mediante la herramienta `QTools` demostró que el modelo de 8 bits consume apenas $\sim 3{,}0\ \mu$J por clasificación.

Si bien el modelo de 4 bits reduciría aún más el consumo en los multiplicadores de hardware y en los buses de lectura de memoria, esta ganancia energética penaliza la tasa de clasificación. Por lo tanto, el análisis previo determina que el formato de **8 bits** es el punto de operación óptimo.